Relatório Final — Etapa 2

Operação, interface e desempenho do banco de dados

Disciplina	Laboratório de Banco de Dados — GPE17M40083 (UCB, 2026/2)
Professor	Samuel Novais Moura Júnior
Tema	Sistema de Gestão de Eventos Acadêmicos (MySQL 8.0)
Grupo	10 — Isaac Barbosa da Silva, Átila de Melo Cantoara, Victor Lemos de Abreu Oliveira, João Pedro Carneiro Ibiapino, Jayme Bernardino da Silva Neto
Entrega	22/11/2026 (AVA) | Apresentação: 23/11/2026, aula 17

RASCUNHO PARA REVISÃO DO GRUPO. Todas as medições e saídas deste documento foram geradas pelo assistente de IA em um **ambiente de teste** (MySQL 8.0.46, Ubuntu 24.04, Python 3.12) com dados 100% sintéticos, usando os scripts do repositório. **O grupo deve reexecutar cada script no próprio ambiente, conferir/substituir os números e prints, e preencher os trechos marcados com [PREENCHER]** (devolutiva da Etapa 1, prints da aplicação e declaração de uso de IA). Remova esta nota antes de entregar.

0. Etapa 1 revisada (correções da devolutiva)

[PREENCHER] Reúna aqui a Etapa 1 revisada (A1 a A5, documentos já presentes em `docs/`) e liste cada observação da devolutiva do professor, o que foi alterado e onde. Equipes que reentregam sem revisão perdem pontos.

Observação do professor	O que foi alterado	Onde (arquivo/seção)
[PREENCHER]	[PREENCHER]	[PREENCHER]
[PREENCHER]	[PREENCHER]	[PREENCHER]

Mudanças no modelo físico desde a Etapa 1 (para o 'o que mudou' da apresentação): adicionadas as tabelas `usuario_app` (login da aplicação, com `senha_hash`) e `auditoria` (gatilhos), 4 visões (`vw_evento_status_atual`, `vw_relatorio_eventos`, `vw_agenda_publica`, `vw_validacao_certificado`), 4 papéis MySQL e 3 índices de apoio aos estudos de desempenho.

1. Aplicação com interface (B1)

Aplicação web em **Python/Flask** conectada ao banco da Etapa 1 (código em `app/`). A interface é simples de propósito: o objeto de avaliação é o que ela revela sobre o banco.

Requisito B1	Como foi atendido	Onde
CRUD de 4+ entidades	5 CRUDs: instituição, sala, categoria de evento, evento e atividade (criar, listar, alterar, excluir)	/crud/<entidade>
2 telas de consulta parametrizada	Ocupação das atividades (JOIN + LEFT JOIN + GROUP BY + HAVING) e Participação por instituição (LEFT JOIN + GROUP BY + HAVING + LIKE)	/consulta/ocupacao, /consulta/participacao
Relatório consolidado por visão	Uma linha por evento, alimentada pela visão `vw_relatorio_eventos`	/relatorio
Operação em 2+ tabelas numa transação	**Encerrar evento e emitir certificados**: grava `historico_status_evento` e `certificado` em uma só transação	/operacoes/encerrar-evento
Erro de restrição compreensível	Mensagens traduzidas por nome de restrição (`db.py`): UNIQUE, CHECK, FK, permissão negada; nunca a exceção do driver	todas as telas

[PREENCHER] Inserir prints: login, lista e formulário de um CRUD, as 2 consultas com filtros, o relatório e uma mensagem de erro tratada (ex.: sala com capacidade 0 -> “A capacidade da sala deve ser maior que zero.”).

Teste automático (`app/teste_fumaca.py`): 33 verificações cobrindo login, CSRF, CRUD, erros amigáveis, consultas, SQL injection, transações e perfis; resultado no ambiente de teste: **33/33 passaram** (saída em `docs/evidencias/06_teste_fumaca.txt`).

2. Camada de acesso a dados (B2)

- **Consultas parametrizadas:** todo valor vindo do usuário é enviado ao driver em uma tupla de parâmetros (%s); não há concatenação de strings com valores. Identificadores de tabela/coluna nos CRUDs vêm de `entidades.py` (fixos), nunca da requisição.
- **Credenciais fora do código:** lidas de variáveis de ambiente/`.env` (no `.gitignore`); o repositório traz só `.env.example`.
- **Uma conta MySQL por perfil**, escolhida em `db.conectar()` pelo perfil do login.

Demonstração de SQL injection resistida (tela “Participação por instituição”). Entradas usadas e resultado obtido:

```
EVIDENCIA B2 - SQL injection resistida (aplicacao Flask, consultas parametrizadas)

[busca normal (sigla UT)] GET /consulta/participacao?instituicao=UT&minimo=0
  HTTP 200 | linhas retornadas: 3 | 'Nenhum resultado': False | excecao do driver exibida: False

[INJECAO 1 (tautologia)] GET /consulta/participacao?instituicao=%27+OR+%271%27%3D%271&minimo=0
  HTTP 200 | linhas retornadas: 0 | 'Nenhum resultado': True | excecao do driver exibida: False

[INJECAO 2 (UNION)] GET /consulta/participacao?instituicao=%27+UNION+SELECT+email%2C1%2C1%2C1+FROM+pessoa--+&minimo=
  HTTP 200 | linhas retornadas: 0 | 'Nenhum resultado': True | excecao do driver exibida: False

[INJECAO 3 (comando destrutivo)] GET /consulta/participacao?tipo=ALUNO%27%3B+DROP+TABLE+instituicao%3B--&minimo=0
  HTTP 200 | linhas retornadas: 0 | 'Nenhum resultado': True | excecao do driver exibida: False
```

```
Tabela instituicao existe e manteve 1 linha(s) (antes: 1). O texto digitado foi tratado como DADO, nunca como SQL.
Trecho do codigo (app.py): WHERE (%s IS NULL OR i.sigla LIKE CONCAT('%%', %s, '%%'))  -> valores enviados em tupla d
```

Leitura: a busca normal devolve 3 linhas; as três tentativas (tautologia `' OR '1'='1`, `UNION SELECT` e `'; DROP TABLE`) foram tratadas como texto literal: nenhuma linha, nenhum erro exibido e a tabela continuou existindo.

3. Transações e controle de concorrência (B3)

3.1 Operação transacional documentada

`/operacoes/encerrar-evento` executa, em uma transação explícita: `START TRANSACTION` -> `SELECT … FOR UPDATE` no evento -> valida status atual -> `INSERT` em `historico_status_evento` (status ENCERRADO) -> `INSERT … SELECT` em `certificado` para quem tem presença mínima -> `COMMIT`. Qualquer erro dispara `ROLLBACK`. A caixa “Simular falha” provoca uma exceção depois dos inserts para mostrar o desfazer.

3.2 Sucesso, ROLLBACK e falha provocada no meio (SQL)

```
EVIDENCIA B3 - transacao multitabela: ROLLBACK, falha no meio, COMMIT (evento 1; tabelas historico_status_evento e c

== Estado inicial
historico  certificados
2  1

== 1) ROLLBACK provocado
momento  historico
dentro da transacao  3
historico  certificados
2  1

== 2) FALHA NO MEIO: 1o INSERT ok, 2o INSERT viola FK (participante inexistente) -> ROLLBACK
ERROR 1452 (23000) at line 3: Cannot add or update a child row: a foreign key constraint fails (`eventos_academicos`
-- (a conexao encerrou sem COMMIT: nada foi gravado) estado depois:
historico  certificados
2  1

== 3) SUCESSO: COMMIT
historico  certificados
3  2
```

Leitura: (1) dentro da transação o histórico passa de 2 para 3 linhas; após o `ROLLBACK` volta a 2. (2) O primeiro INSERT funciona, o segundo viola `fk_certificado_participante` (erro 1452); como não houve COMMIT, nada permaneceu (2 e 1). (3) No sucesso, as duas tabelas avançam juntas (3 e 2): atomicidade das duas tabelas.

3.3 Experimento de concorrência com duas sessões

Nível de isolamento em vigor: **REPEATABLE READ** (padrão do InnoDB), `innodb_lock_wait_timeout` = 50 s.

```
################ EXPERIMENTO 1: BLOQUEIO / ESPERA (SELECT ... FOR UPDATE vs UPDATE na mesma linha)
-- Sessao C (observador) durante a espera: quem bloqueia quem
waiting_pid  blocking_pid  wait_age_secs  consulta_esperando  locked_type  locked_index
49  48  3  UPDATE atividade SET vagas=vagas WHERE id_atividade=1  RECORD  PRIMARY

sessao_A  hora
A: iniciou transacao  2026-10-27 02:05:30.432
vagas
30
sessao_A  hora
A: travou a linha da atividade 1, vai segurar 6 s  2026-10-27 02:05:30.432
sessao_A  hora
A: COMMIT (libera o bloqueio)  2026-10-27 02:05:36.433

sessao_B  hora
B: vai tentar o UPDATE  2026-10-27 02:05:31.935
sessao_B  hora
B: UPDATE concluiu  2026-10-27 02:05:36.434

################ EXPERIMENTO 2: TIMEOUT DE ESPERA (erro 1205)
B_tenta_em
2026-10-27 02:05:37.451
ERROR 1205 (HY000) at line 1: Lock wait timeout exceeded; try restarting transaction

################ EXPERIMENTO 3: DEADLOCK (ordem inversa de travas: atividades 1 e 2)
--- saida da sessao A:
passo  hora
A travou atividade 1  2026-10-27 02:05:42.453
passo
A conseguiu atividade 2
--- saida da sessao B:
```

```
passo  hora
B travou atividade 2  2026-10-27 02:05:43.455
ERROR 1213 (40001) at line 1: Deadlock found when trying to get lock; try restarting transaction

--- SHOW ENGINE INNODB STATUS (secao do ultimo deadlock detectado):
LATEST DETECTED DEADLOCK
*** (1) TRANSACTION:
TRANSACTION 10278, ACTIVE 4 sec starting index read
UPDATE atividade SET vagas=vagas WHERE id_atividade=2
*** (1) HOLDS THE LOCK(S):
RECORD LOCKS space id 206 page no 5 n bits 240 index PRIMARY of table `eventos_academicos`.`atividade` trx id 10278
*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 206 page no 5 n bits 240 index PRIMARY of table `eventos_academicos`.`atividade` trx id 10278
*** (2) TRANSACTION:
TRANSACTION 10279, ACTIVE 3 sec starting index read
UPDATE atividade SET vagas=vagas WHERE id_atividade=1
*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 206 page no 5 n bits 240 index PRIMARY of table `eventos_academicos`.`atividade` trx id 10279
*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 206 page no 5 n bits 240 index PRIMARY of table `eventos_academicos`.`atividade` trx id 10279
*** WE ROLL BACK TRANSACTION (2)

################ EXPERIMENTO 4: NIVEL DE ISOLAMENTO (leitura nao repetivel)
== Sessao A em REPEATABLE READ; Sessao B faz UPDATE (autocommit) no meio da transacao de A
sessao_B
B: UPDATE vagas+5 (commit imediato)
A_isolamento
REPEATABLE-READ
A_leitura_1
30
A_leitura_2
30
== Sessao A em READ COMMITTED; Sessao B faz UPDATE (autocommit) no meio da transacao de A
sessao_B
B: UPDATE vagas+5 (commit imediato)
A_isolamento
READ-COMMITTED
A_leitura_1
30
A_leitura_2
35
```

Leitura dos resultados. Experimento 1 (bloqueio): a sessão A travou a linha da atividade 1 (`FOR UPDATE`) às 02:05:30,4 e deu COMMIT às 02:05:36,4; a sessão B tentou o `UPDATE` às 02:05:31,9 e só concluiu às 02:05:36,4, ou seja, **esperou cerca de 4,5 s**. A visão `sys.innodb_lock_waits` mostrou a sessão 49 esperando pela 48. **Experimento 2:** com timeout de 2 s a sessão B falhou com **erro 1205**. **Experimento 3 (deadlock):** A e B travaram as atividades 1 e 2 em ordem inversa; o InnoDB detectou o ciclo e desfez uma delas (**erro 1213**, “WE ROLL BACK TRANSACTION (2)”), a outra concluiu. **Experimento 4:** sob REPEATABLE READ, A leu 30 nas duas leituras; sob READ COMMITTED leu 30 e depois 35 (*leitura não repetível*).

3.4 Concorrência pela aplicação: duas inscrições na última vaga

```
EVIDENCIA B3 - concorrencia pela APLICACAO: duas inscricoes simultaneas na ULTIMA vaga
Atividade 8197 com vagas=1. Participantes 65543 (usuario 1) e 65544 (usuario 2).
  Usuario 1 (segura o bloqueio 4 s): enviou em t= 0.3s, terminou em t= 4.4s
  Usuario 2 (chega 1 s depois): enviou em t= 1.4s, terminou em t= 4.4s

Resultado final no banco (status de cada inscricao):
   {'id_participante': 65543, 'status_inscricao': 'CONFIRMADA', 'hora': datetime.timedelta(seconds=7614)}
   {'id_participante': 65544, 'status_inscricao': 'LISTA_ESPERA', 'hora': datetime.timedelta(seconds=7614)}
Confirmadas: 1 (limite 1). Sem o SELECT ... FOR UPDATE as duas poderiam ler 0 confirmadas e ambas ficariam CONFIRMAD
```

Leitura: o usuário 2 enviou a requisição 1 s depois e ficou bloqueado até o usuário 1 terminar (t = 4,4 s). Ao entrar, enxergou 1 confirmada e foi para a **LISTA_ESPERA**: o limite de vagas não foi ultrapassado.

3.5 Justificativa do nível de isolamento

Adotamos **REPEATABLE READ** (padrão) combinado com `SELECT … FOR UPDATE` nos pontos críticos (controle de vagas e encerramento de evento). O experimento 4 mostra que READ COMMITTED permitiria que a contagem de confirmadas mudasse entre a leitura e o INSERT; o `FOR UPDATE` serializa só as inscrições da mesma atividade, sem o custo de bloqueio amplo do SERIALIZABLE. Custo assumido: inscrições simultâneas na mesma atividade esperam em fila (poucos segundos) e, em ordem inversa de travas, pode ocorrer deadlock (erro 1213) — por isso a aplicação trata 1205/1213 com mensagem para tentar de novo.

4. Otimização de consultas (B4)

Volume: 120.009 inscrições, 50.009 pessoas, 6.005 atividades, 6.002 sessões (carga sintética `sql/08_carga_volume.sql`). Método: `EXPLAIN ANALYZE` 3 vezes por medida; mediana do tempo do nó raiz.

Estudo (consulta crítica)	Índice criado	Antes (ms)	Depois (ms)	Ganho	Custo em espaço
1. Inscrições por status num intervalo de datas	idx_inscricao_data (data_inscricao, status_inscricao)	61,1	17,9	71%	+5,5 MB
2. Busca de pessoa por prefixo do nome (LIKE 'Ana S%')	idx_pessoa_nome (nome)	10,6	2,35	78%	+2,5 MB
3. Sessões de uma sala em um período, ordenadas	idx_sessao_sala_inicio (id_sala, inicio)	3,55	0,95	73%	~ 0 (tabela pequena)

```
EVIDENCIA B4 - estudos de EXPLAIN ANALYZE (MySQL 8.0, dados sinteticos de 08_carga_volume.sql)
Cada medida = EXPLAIN ANALYZE executado 3 vezes; reporta-se a MEDIANA do tempo do no raiz (ms).

================================================================================================
ESTUDO 1 - inscricoes em um intervalo de datas
Consulta: SELECT status_inscricao, COUNT(*) AS total FROM inscricao_atividade WHERE data_inscricao BETWEEN '2025-01-
Volume: 120009 linhas em inscricao_atividade

-- ANTES do indice (tempos: [61.1, 60.6, 63.3] ms; mediana 61.1 ms) | tabela: 10.52 MB dados, 14.03 MB indices
Plano:
   -> Table scan on <temporary>  (actual time=61.1..61.1 rows=3 loops=1)
       -> Aggregate using temporary table  (actual time=61.1..61.1 rows=3 loops=1)
           -> Filter: (inscricao_atividade.data_inscricao between '2025-01-01' and '2025-06-30 23:59:59')  (cost=116
               -> Table scan on inscricao_atividade  (cost=11681 rows=115131) (actual time=0.208..23.2 rows=120009 l

-- DEPOIS: CREATE INDEX idx_inscricao_data ON inscricao_atividade (data_inscricao, status_inscricao)
   (tempos: [18.9, 17.6, 17.9] ms; mediana 17.9 ms) | tabela: 10.52 MB dados, 19.55 MB indices (+5.52 MB)
Plano:
   -> Table scan on <temporary>  (actual time=18.9..18.9 rows=3 loops=1)
       -> Aggregate using temporary table  (actual time=18.9..18.9 rows=3 loops=1)
           -> Filter: (inscricao_atividade.data_inscricao between '2025-01-01' and '2025-06-30 23:59:59')  (cost=926
               -> Covering index range scan on inscricao_atividade using idx_inscricao_data over ('2025-01-01 00:00:

>> Resultado: 61.1 ms -> 17.9 ms (reducao de 71%).
================================================================================================
ESTUDO 2 - busca de pessoa pelo inicio do nome (LIKE 'Ana S%')
Consulta: SELECT id_pessoa, nome, email FROM pessoa WHERE nome LIKE 'Ana S%'
Volume: 50009 linhas em pessoa

-- ANTES do indice (tempos: [11.8, 10.6, 10.5] ms; mediana 10.6 ms) | tabela: 4.52 MB dados, 3.52 MB indices
Plano:
   -> Filter: (pessoa.nome like 'Ana S%')  (cost=4995 rows=5469) (actual time=0.0327..11.8 rows=1250 loops=1)
       -> Table scan on pessoa  (cost=4995 rows=49227) (actual time=0.024..8.78 rows=50009 loops=1)

-- DEPOIS: CREATE INDEX idx_pessoa_nome ON pessoa (nome)
   (tempos: [3.2, 2.35, 2.16] ms; mediana 2.35 ms) | tabela: 4.52 MB dados, 6.03 MB indices (+2.51 MB)
Plano:
   -> Index range scan on pessoa using idx_pessoa_nome over ('Ana S' <= nome <= 'Ana S??????????????????????????????

>> Resultado: 10.6 ms -> 2.35 ms (reducao de 78%).
================================================================================================
ESTUDO 3 - sessoes de uma sala em um periodo
Consulta: SELECT id_atividade, numero_sessao, inicio, fim FROM sessao_atividade WHERE id_sala = 1 AND inicio BETWEEN
Volume: 6002 linhas em sessao_atividade

-- ANTES do indice (tempos: [3.55, 3.53, 3.62] ms; mediana 3.55 ms) | tabela: 0.31 MB dados, 0.42 MB indices
Plano:
   -> Sort: sessao_atividade.inicio  (cost=65.7 rows=3002) (actual time=3.53..3.55 rows=508 loops=1)
       -> Filter: (sessao_atividade.inicio between '2025-07-01' and '2025-12-31 23:59:59')  (cost=65.7 rows=3002) (a
           -> Index lookup on sessao_atividade using fk_sessao_sala (id_sala=1)  (cost=65.7 rows=3002) (actual time=

-- DEPOIS: CREATE INDEX idx_sessao_sala_inicio ON sessao_atividade (id_sala, inicio)
   (tempos: [0.992, 0.947, 0.887] ms; mediana 0.947 ms) | tabela: 0.31 MB dados, 0.42 MB indices (+0.00 MB)
Plano:
   -> Index range scan on sessao_atividade using idx_sessao_sala_inicio over (id_sala = 1 AND '2025-07-01 00:00:00'

>> Resultado: 3.55 ms -> 0.947 ms (reducao de 73%).
================================================================================================
CONTRAEXEMPLO (ESTUDO 2): LIKE com curinga no inicio NAO usa o indice como busca:
   mediana 18.8 ms; plano:
   -> Filter: (pessoa.nome like '%Souza%')  (cost=5002 rows=5477) (actual time=5.33..23.1 rows=5001 loops=1)
       -> Covering index scan on pessoa using idx_pessoa_nome  (cost=5002 rows=49301) (actual time=0.0892..11.4 rows
```

Leitura dos planos. Estudo 1: antes, *Table scan* nas 120 mil linhas e filtro depois da leitura; depois, *Covering index range scan*: só a faixa de datas é lida (21 mil entradas) e, como o índice inclui `status_inscricao`, a tabela nem é consultada. A data vem primeiro porque o filtro é um *intervalo* (colunas depois de um intervalo não restringem a busca); o status vem em seguida só para cobrir o `GROUP BY`. **Estudo 2:** o *LIKE* com prefixo fixo vira *Index range scan* sobre a faixa 'Ana S…'. **Estudo 3:** antes, usava o índice da FK (`fk_sessao_sala`), lia as 3.002 sessões da sala, filtrava as datas e ordenava (*Sort*); com (id_sala, inicio) a igualdade vem primeiro e o intervalo depois, o índice já entrega as linhas ordenadas e o *Sort* desaparece.

Discussão honesta. (a) O contraexemplo mostra que `LIKE '%Souza%'` não se beneficia do índice como busca: o plano varre o índice inteiro (~18,8 ms), pior que a busca por prefixo; seria preciso índice FULLTEXT ou outra estratégia. (b) O ganho do estudo 3 é grande em proporção, mas pequeno em valor absoluto (~2,6 ms) porque a tabela tem só 6 mil linhas; só importaria em volume muito maior. (c) Todo índice custa espaço e deixa INSERT/UPDATE um pouco mais lentos: o índice do estudo 1 ocupa 5,5 MB para 120 mil linhas. (d) Os tempos mudam entre máquinas e execuções; as medianas acima são do ambiente de teste. [PREENCHER] substituir pelos números medidos pelo grupo.

5. Segurança e controle de acesso (B5)

Papel / conta	Pode	Não pode (comprovado)
r_admin_eventos / usr_admin_eventos	CRUD em todas as tabelas do domínio; inserir/atualizar usuários; ler auditoria	apagar auditoria; DROP TABLE (sem DDL)
r_organizador / usr_organizador	Ler o domínio; inserir/atualizar eventos, atividades, inscrições, histórico, certificados (só INSERT); apagar sessões	alterar certificado (`REVOKE UPDATE`); apagar evento; ler usuario_app e auditoria
r_consulta / usr_consulta	Ler só `vw_agenda_publica` e `vw_validacao_certificado`	ler qualquer tabela (inclui pessoa) e o relatório
r_autenticacao / usr_app_auth	Ler login, hash, perfil e ativo; atualizar `ultimo_login`	ler qualquer dado do domínio; alterar perfil

Princípio do menor privilégio: privilégios por tabela (e por coluna no perfil de autenticação); `GRANT` e `REVOKE` em `sql/06_seguranca.sql`. As contas com senha ficam em arquivo local fora do Git. **Evidência com login real** (um comando permitido e um negado por perfil):

```
EVIDENCIA B5 - perfis de acesso com login real (MySQL 8.0)
Contas: usr_consulta, usr_organizador, usr_admin_eventos, usr_app_auth

######## usr_consulta (papel r_consulta)

>>> [usr_consulta] SELECT evento, atividade, sala FROM vw_agenda_publica LIMIT 3
evento  atividade  sala
Semana de BD  Oficina SQL  Sala 101
Evento sintético 1  Atividade sintética 2 do evento 3  Sala 101
Evento sintético 2  Atividade sintética 3 do evento 4  Sala 101

>>> [usr_consulta] SELECT * FROM vw_validacao_certificado
codigo_validacao  titular_mascarado  evento  carga_horaria  emitido_em
ABC-123  Maria L.  Semana de BD  4.0  2026-11-13 10:00:00

>>> [usr_consulta] SELECT email FROM pessoa LIMIT 1
ERROR 1142 (42000) at line 1: SELECT command denied to user 'usr_consulta'@'localhost' for table 'pessoa'

>>> [usr_consulta] SELECT * FROM vw_relatorio_eventos LIMIT 1
ERROR 1142 (42000) at line 1: SELECT command denied to user 'usr_consulta'@'localhost' for table 'vw_relatorio_event

######## usr_organizador (papel r_organizador)

>>> [usr_organizador] SELECT COUNT(*) AS eventos FROM evento
eventos
2002

>>> [usr_organizador] UPDATE certificado SET carga_horaria=99 WHERE id_certificado=1
ERROR 1142 (42000) at line 1: UPDATE command denied to user 'usr_organizador'@'localhost' for table 'certificado'

>>> [usr_organizador] DELETE FROM evento WHERE id_evento=2
ERROR 1142 (42000) at line 1: DELETE command denied to user 'usr_organizador'@'localhost' for table 'evento'

>>> [usr_organizador] SELECT * FROM usuario_app
ERROR 1142 (42000) at line 1: SELECT command denied to user 'usr_organizador'@'localhost' for table 'usuario_app'

>>> [usr_organizador] SELECT * FROM auditoria LIMIT 1
ERROR 1142 (42000) at line 1: SELECT command denied to user 'usr_organizador'@'localhost' for table 'auditoria'

######## usr_admin_eventos (papel r_admin_eventos)

>>> [usr_admin_eventos] SELECT COUNT(*) AS auditorias FROM auditoria
auditorias
0

>>> [usr_admin_eventos] DELETE FROM auditoria
ERROR 1142 (42000) at line 1: DELETE command denied to user 'usr_admin_eventos'@'localhost' for table 'auditoria'

>>> [usr_admin_eventos] DROP TABLE sala
ERROR 1142 (42000) at line 1: DROP command denied to user 'usr_admin_eventos'@'localhost' for table 'sala'

######## usr_app_auth (papel r_autenticacao: so login)

>>> [usr_app_auth] SELECT * FROM evento LIMIT 1
```

```
ERROR 1142 (42000) at line 1: SELECT command denied to user 'usr_app_auth'@'localhost' for table 'evento'

>>> [usr_app_auth] SELECT login, perfil FROM usuario_app LIMIT 1

######## SHOW GRANTS (papeis)
r_admin_eventos@%
GRANT USAGE ON *.* TO `r_admin_eventos`@`%`
GRANT SELECT, INSERT, UPDATE, DELETE ON `eventos_academicos`.`atividade` TO `r_admin_eventos`@`%`
```

Visões de restrição: `vw_agenda_publica` restringe *colunas* (sem e-mail, telefone, nascimento nem inscritos) e *linhas* (omite eventos cujo status atual é CANCELADO); `vw_validacao_certificado` mostra o titular mascarado (“Maria L.”). As visões usam `SQL SECURITY DEFINER`, por isso o perfil de consulta lê a visão sem ter acesso às tabelas.

Senhas: bcrypt (custo 12) com sal aleatório embutido no hash (`$2b$12$…`), gerado pela aplicação; a tabela `usuario_app` recusa (CHECK `ck_usuario_app_hash`) qualquer valor que não pareça um hash, barrando senha em texto claro por engano. [PREENCHER] print de `SELECT login, LEFT(senha_hash, 15) FROM usuario_app`.

Dados pessoais do projeto e cuidados (LGPD):

Dado	Tabela	Cuidado adotado
Nome, e-mail, telefone, data de nascimento	pessoa	Carga 100% fictícia (sem dados reais de terceiros); leitura restrita a admin/organizador; visões públicas não expõem
Login e senha	usuario_app	Só hash bcrypt com sal; perfil de autenticação enxerga apenas as colunas necessárias
Autor das alterações	auditoria	Somente leitura para admin; escrita apenas pelos gatilhos
Nome do titular do certificado	certificado	Validação pública mostra o nome mascarado

Observação: o modelo não guarda CPF. Fora do escopo da disciplina: HTTPS, limite de tentativas de login e criptografia em repouso (citar como limitações).

6. Gatilhos e auditoria (B6)

Justificativa pelo domínio. (1) `trg_aud_inscricao_atividade_upd`: audita *status* e *presença* da inscrição, que determinam quem tem direito a certificado; cancelar uma inscrição ou alterar a presença muda esse direito. (2) `trg_aud_evento_upd`: audita *datas, modalidade, limite e valor* do evento, informações que afetam os inscritos depois da abertura das inscrições. Mudanças em colunas não relevantes (ex.: descrição) não geram registro.

Autor: a aplicação executa `SET @usuario_app = '<login>'` ao conectar; o gatilho grava `COALESCE(@usuario_app, USER())`. Limitação: a variável de sessão identifica o usuário da aplicação, mas não é prova criptográfica.

```
EVIDENCIA B6 - gatilhos de auditoria
linhas_auditoria_antes
0
-- UPDATE 1 (inscricao: status), UPDATE 2 (evento: data), UPDATE 3 (evento: so descricao = nao auditado)
linhas_auditoria_depois
2
*************************** 1. row ***************************
   id_auditoria: 1
 tabela_afetada: inscricao_atividade
 chave_registro: participante=1;atividade=1
          autor: ana.organizadora
      data_hora: 27/10/2026 02:05:15
 valor_anterior: {"status_inscricao": "CONFIRMADA", "percentual_presenca": null}
valor_posterior: {"status_inscricao": "CANCELADA", "percentual_presenca": null}
*************************** 2. row ***************************
   id_auditoria: 2
 tabela_afetada: evento
 chave_registro: evento=1
          autor: ana.organizadora
      data_hora: 27/10/2026 02:05:15
 valor_anterior: {"data_fim": "2026-11-12 18:00:00.000000", "modalidade": "PRESENCIAL", "data_inicio": "2026-11-10 0
valor_posterior: {"data_fim": "2026-11-12 18:00:00.000000", "modalidade": "PRESENCIAL", "data_inicio": "2026-11-11 0
-- sem @usuario_app o autor vira o usuario do banco:
autor
root@localhost
```

Leitura: 3 UPDATEs geraram **2** linhas de auditoria (a alteração só de descrição foi ignorada); cada linha traz autor (*ana.organizadora*), data/hora e JSON com os valores anterior e posterior. Sem `@usuario_app` o autor passa a ser o usuário do banco.

7. Projeto físico e armazenamento (B7)

7.1 Revisão dos tipos de dados

Coluna / decisão	O que mudaria	Por quê
Chaves `BIGINT UNSIGNED` (pessoa, evento, atividade…)	Avaliar `INT UNSIGNED` (até ~4,29 bilhões)	O volume projetado (seção 7.2) está muito abaixo; BIGINT gasta 4 bytes a mais por linha e em cada índice/FK (a tabela de inscrições tem PK com 2 colunas BIGINT). Mantido por simplicidade; trocar exigiria recriar FKs.
`criado_em TIMESTAMP` (pessoa, usuario_app)	Trocar por `DATETIME`	TIMESTAMP só vai até 2038 e depende do fuso; as demais datas do modelo já são DATETIME.
Status/tipos como `VARCHAR + CHECK`	Manter (não usar ENUM)	ENUM economiza bytes, mas alterar valores exige ALTER TABLE; VARCHAR+CHECK mantém a regra visível no DDL e portátil.
`email VARCHAR(160)` com UNIQUE, collation `utf8mb4_0900_ai_ci`	Manter	Índice único (640 bytes máx.) cabe no limite do InnoDB; collation sem distinção de acento/caixa evita e-mails duplicados por maiúsculas.
`carga_horaria DECIMAL(4,1)` (atividade) e `DECIMAL(5,1)` (certificado)	Manter	O certificado soma atividades, por isso tem 1 dígito a mais; nenhuma perda de precisão (DECIMAL, não FLOAT).
Booleanos (`BOOLEAN`)	Manter	TINYINT(1): 1 byte, com valores ambíguos evitados por NOT NULL DEFAULT.

7.2 Estimativa de crescimento em 2 anos

Tamanho por linha medido (dados + índices) no ambiente de teste × hipóteses de uso. **[PREENCHER]** ajustar as hipóteses ao uso real esperado.

```
1) Tamanho medido (dados + indices) no banco com carga sintetica:
tabela                        linhas  dados MB  indices MB  bytes/linha
pessoa                         48646      4.52        6.03          227
participante                   50628      2.52        2.78          110
evento                          2006      0.20        0.22          221
historico_status_evento         2008      0.12        0.08          106
atividade                       6004      1.52        0.16          292
sessao_atividade                6002      0.31        0.42          128
inscricao_atividade           111191     10.52       19.55          284
certificado                        3      0.02        0.05          n/d
auditoria                          4      0.02        0.02          n/d

2) Hipoteses de crescimento (AJUSTE aos numeros reais esperados pelo grupo):
   eventos_ano = 120
   ativ_por_evento = 3
   insc_por_ativ = 40
   cert_por_evento = 60
   pessoas_novas_ano = 2000
   aud_por_insc = 0.5

3) Projecao em 2 anos (linhas e tamanho = linhas x bytes/linha medido; nao inclui binlog nem backups):
tabela                      linhas (2 anos)   tamanho MB
pessoa                                4,000          0.9
participante                          4,000          0.4
evento                                  240          0.1
historico_status_evento                 720          0.1
atividade                               720          0.2
sessao_atividade                      1,080          0.1
inscricao_atividade                  28,800          7.8
certificado                          14,400          3.0
auditoria                            14,400          4.8
TOTAL                                               17.4   (~0.02 GB)

Tabelas de maior movimento: inscricao_atividade e auditoria (>95% das linhas). Candidatas a particionamento/arquivam
se passarem de dezenas de milhoes de linhas; no volume projetado acima o crescimento e administravel sem particionar
```

7.3 Plano de backup

Item	Decisão
O que copiar	Banco `eventos_academicos` completo (dados, visões, gatilhos, rotinas) com `mysqldump --single-transaction --routines --triggers --events`. Contas MySQL e GRANTs são recriados por `06_seguranca.sql` + `06_usuarios.local.sql`; `.env` e `backup.cnf` ficam fora do Git, em gerenciador de senhas.
Frequência	Completo diário às 02:00 (baixo movimento). Se a perda aceitável for menor que 24 h, somar backup incremental por binlog.
Retenção	14 dias de diários, 8 semanais, 12 mensais (o script aplica a janela diária; as demais por rotação).
Onde guardar	Regra 3-2-1: 3 cópias, 2 mídias, 1 fora do local (disco do servidor + outra máquina/nuvem).
Teste de restauração	`restaurar_teste.sh` restaura o último dump em banco temporário e compara a contagem de cada tabela e de gatilhos; executar após mudança de esquema e ao menos mensalmente.

```
EVIDENCIA B7 - backup (mysqldump) e teste de restauracao em banco temporario
Backup OK: /tmp/dumps_teste/eventos_academicos_20261027_020731.sql.gz (1.9M)

Restaurando /tmp/dumps_teste/eventos_academicos_20261027_020731.sql.gz em eventos_academicos_restore_test ...
atividade                   original=6006     restaurado=6006     ok
auditoria                   original=4        restaurado=4        ok
categoria_evento            original=2        restaurado=2        ok
certificado                 original=3        restaurado=3        ok
evento                      original=2006     restaurado=2006     ok
historico_status_evento     original=2008     restaurado=2008     ok
inscricao_atividade         original=120009   restaurado=120009   ok
instituicao                 original=1        restaurado=1        ok
organizacao_evento          original=0        restaurado=0        ok
organizador                 original=0        restaurado=0        ok
participante                original=50009    restaurado=50009    ok
pessoa                      original=50009    restaurado=50009    ok
sala                        original=2        restaurado=2        ok
sessao_atividade            original=6002     restaurado=6002     ok
usuario_app                 original=3        restaurado=3        ok
Gatilhos no restaurado: 2 (original: 2)
RESTAURAÇÃO VERIFICADA
```

Leitura: o backup compactado (1,9 MB) foi restaurado em um banco temporário; as 15 tabelas e os 2 gatilhos conferem com o original (“RESTAURAÇÃO VERIFICADA”).

7.4 Particionamento e arquivamento

Candidatas pelo volume projetado: `inscricao_atividade` e `auditoria`. **Restrição técnica importante:** o InnoDB não permite chaves estrangeiras em tabelas particionadas; como `inscricao_atividade` tem FKs, a proposta para ela é **arquivamento** (mover inscrições de eventos ENCERRADOS há mais de 2 anos para uma tabela `inscricao_atividade_arquivo`, preservando certificados), não particionamento. A `auditoria` não tem FK e pode ser particionada por `RANGE` em `data_hora` (por ano), exigindo que a PK inclua a coluna de partição, ex.: `PRIMARY KEY (id_auditoria, data_hora)`. No volume projetado (poucas dezenas de milhares de linhas em 2 anos) nenhuma das duas é necessária agora; a proposta fica registrada para quando a auditoria passar de milhões de linhas.

8. Declaração de uso de inteligência artificial

Ferramenta: Claude (Anthropic), via claude.ai.

Partes em que foi utilizada: leitura do enunciado e levantamento do que faltava no repositório; revisão do `01_ddl.sql`; geração inicial dos scripts `04_views`, `05_triggers`, `06_seguranca`, `06_usuarios.exemplo`, `07_estudo_indices`, `08_carga_volume` e `09_transacao_concorrencia`; da aplicação Flask (`app/`) e do teste automático; dos scripts de backup/restauração; do esqueleto e deste rascunho de relatório. As execuções e medições que aparecem neste documento foram feitas pelo assistente em ambiente de teste próprio. Na Etapa 1, conforme o README: apoio à estruturação do domínio, revisão de regras, SQL inicial e documentação.

Como o grupo verificou o resultado: [PREENCHER — somente o que o grupo realmente fez. Exemplos do que verificar: executar todos os scripts em base limpa; ler e entender cada gatilho, visão e papel; testar cada perfil com login real; rodar `teste_fumaca.py`; reexecutar os experimentos de concorrência e os EXPLAIN e conferir os números; ajustes feitos pelo grupo].

O que o grupo escreveu/decidiu por conta própria: [PREENCHER].

Nota: na arguição qualquer integrante pode ser questionado sobre qualquer decisão do projeto; saber explicar o próprio trabalho pesa mais do que ter usado uma ferramenta.